# Final Report: The Concurrent Ball Tagging Game

CODE ## **1. Project Overview** This project implements a concurrent simulation of a ball-tagging game where two players (Player A and Player B) compete to color white balls in a rectangular court.

### Multi-Language Architecture

To demonstrate cross-platform concurrency and Inter-Process Communication (IPC), the system is built using three different languages: - **Code A (Player A)**: Implemented in **Java**. - **Code B (Player B)**: Implemented in **C++**. - **Code C (Game Host)**: Implemented in **Python**.

**GitHub Repository**: [INSERT_GITHUB_REPO_LINK_HERE]

---

## 2. Step-by-Step Implementation

The project provides three distinct synchronization solutions, each following these core steps:

### Step 1: Initialization (The Host)

The Python Host defines the court size ($100 \times 100$) and generates $n$ white balls at random coordinates. It opens a TCP socket server to listen for player connections.

### Step 2: Concurrent Launch

The Host launches the Java and C++ processes. Upon connection, the Host sends the initial game state (ball positions and game duration) to both players.

### Step 3: Movement and Discovery (The Players)

Both Player A (Java) and Player B (C++) execute a "Random Walk" loop. In each iteration: 1. They update their $(x, y)$ coordinates. 2. They calculate the Euclidean distance to all $n$ balls. 3. If a ball is within the **capture threshold (5.0 units)**, they attempt to tag it by sending a message to the Host.

### Step 4: Synchronization and State Transition

The Host manages the "Source of Truth." Depending on the chosen solution, it prevents race conditions using one of three paradigms:

### Solution 1: Mutex / Host-Side Locking

- **Process**: The Host maintains a list of `threading.Lock` objects (one per ball).

- **Action**: When a request for ball $i$ arrives, the Host acquires `Lock[i]`. It checks if `Color[i] == WHITE`. If yes, it sets it to the player's color and returns `SUCCESS`. If not, it returns `FAILURE`.

**Solution 2: Atomic CAS Simulation**

- **Process**: The Host treats the socket request-handling as an atomic transaction.
- **Action**: It performs a "Compare-and-Swap" logic: *If current state is 0, update to new color*. Because the Host's internal request handling is synchronized, it ensures that even if two packets arrive nearly simultaneously, one is processed fully before the other.

**Solution 3: Message Passing (Shared-Nothing)**

- **Process**: Players treat the Host as a remote service. They have no direct access to the ball data.
- **Action**: The Host processes requests from a FIFO queue. The first "intent to tag" message for a specific white ball that is dequeued wins the ball. All subsequent messages for that index are discarded.

**Step 5: Termination and Tallying**

After the 2-minute timer expires (or the specified test duration), the Host stops accepting requests, counts the final colors, and declares the winner (Player A, Player B, or Draw).

---

## 3. Discussion and Assignment Questions

**Q1: Speed of each player (equal, different, ..etc)**

In our simulation, players move at similar speeds (step size $\pm 3.0$ units). However, if Player A were given a larger step size or a faster processing loop, it would cover more area per second, leading to a higher probability of winning.

**Q2: What race condition can occur if synchronization is not used?**

Without synchronization, both players could read a ball's state as `WHITE` at the exact same microsecond. Both would then send a write command. The Host might process both, meaning one player's color would be overwritten by the other, and the total count of "captured" events would be inconsistent with the final visual state of the balls.

**Q3: What happens if both players try to color the same ball simultaneously?**

- **Un-synchronized**: A "Lost Update" occurs where the second player's color overwrites the first, even though the ball was no longer white.
- **Synchronized**: The first request to be processed by the Host (via Lock or Queue) succeeds. The second request fails the "Is it White?" check and is rejected.

**Q4: How would the system behave if the number of balls is very large?**

If $n \gg 10^5$: 1. **Memory**: The Host would require significant RAM to store positions and locks. 2. **Performance**: $O(n)$ distance checks per player movement would lag the simulation. 3. **Bottleneck**: The centralized Host would become a bottleneck for TCP requests. Spatial partitioning (dividing the court into zones) would be required.

**Q5: Optimal number of balls required to prevent a draw.**

The number of balls $n$ should be **odd** ($n \equiv 1 \pmod 2$). This ensures that even if all balls are tagged, a 50/50 split is mathematically impossible.

**Q6: How can this game be implemented using message passing?**

This is exactly how our IPC version works! Instead of a shared memory array (C++ pointers or Java references), players send "messages" (TCP packets) to a central "Actor" (the Host). The Actor is the only component that modifies the state.

**Q7: Modeling using two parallel processes.**

We modeled the system using **three** parallel processes (Host + 2 Players). This allows the UI/Host logic to remain responsive while the players use maximum CPU cycles for movement and collision detection.

**Q8: How does concurrency improve the number of balls processed?**

Parallelism allows different areas of the court to be explored simultaneously. In a sequential version, a single player must visit every ball one by one. With two players, the "search space" is effectively split, potentially doubling the capture rate.

**Q9: Define the ideal speedup.**

The ideal speedup for two players is $S = 2.0$. This means the game's objective (tagging all balls) is completed in exactly half the time compared to a single-player run.

**Q10: Why might the actual speedup be less than the ideal value?**

1. **Synchronization Overhead**: Time spent waiting for locks or processing network packets.
2. **Resource Contention**: Both players trying to tag the same ball (wasted effort).
3. **Communication Latency**: The time it takes for a "TAG" request to travel from the Java/C++ process to the Python Host.